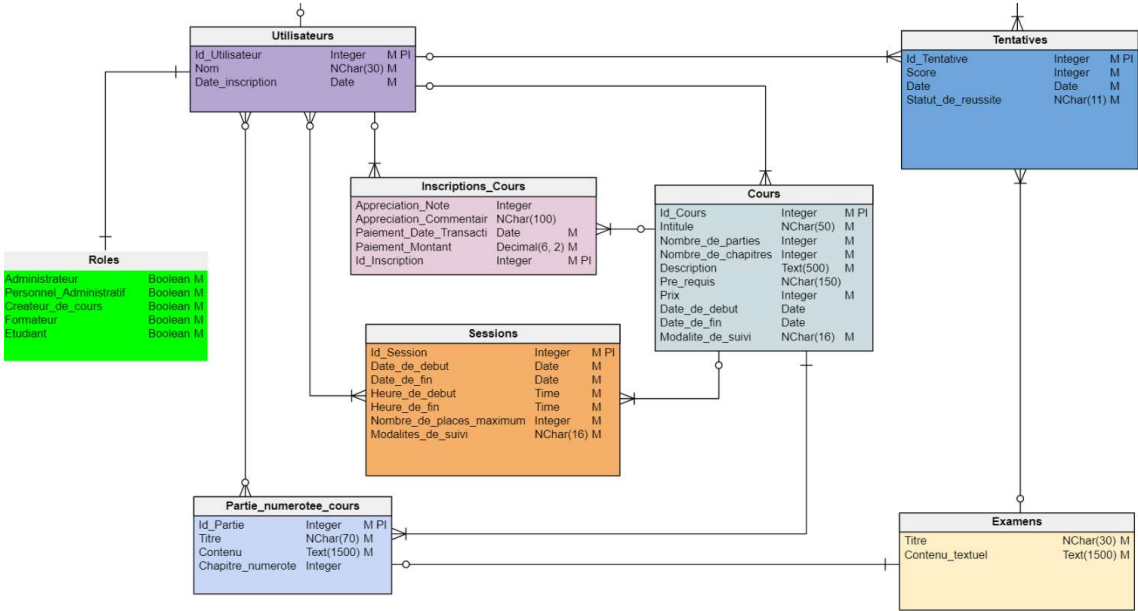
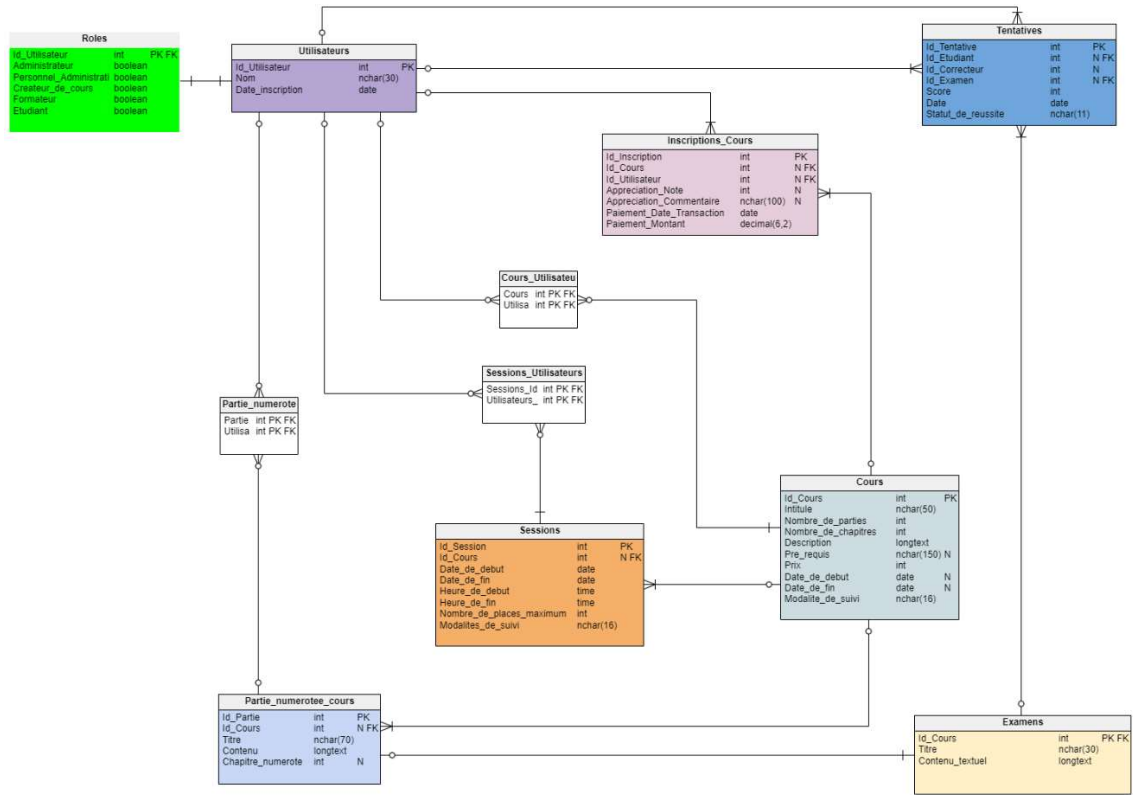


Conception de base de données - RAPPORT

Après lecture du sujet et de nombreux changements, nous avons conçu le modèle Entité-Association suivant sur Vertabelo :



Nous avons ensuite généré le modèle physique correspondant, et y avons ajouté les contraintes en cascade :



Nous avons alors généré le script de création en MySQL correspondant (voir database\_create.sql) :

```
1 -- Created by Vertabelo (http://www.vertabelo.com)
2 -- Last modification date: 2024-04-03 01:13:16.991
3
4 -- Tables
5 -- Table: Cours
6 CREATE TABLE Cours (
7   Id_Cours int NOT NULL COMMENT 'PK
8   Identifiant unique du cours.',
9   Titre varchar(50) NOT NULL COMMENT 'Titre du cours, utilisé pour l''''identifiant.',
10  Nombre_de_parties int NOT NULL COMMENT 'Nombre de parties composant le cours.',
11  Nombre_de_chapitres int NOT NULL COMMENT 'Nombre de chapitres dans le cours.',
12  Description_longtext NOT NULL COMMENT 'Description du cours, fournissant des détails sur son contenu et ses objectifs.',
13  Pre_requis varchar(150) NULL COMMENT 'Prérequis nécessaires pour suivre le cours.',
14  Prix int NOT NULL COMMENT 'Prix du cours, s''''il est applicable.',
15  Date_de_debut date NULL COMMENT 'Date de début du cours, si elle est définie.',
16  Date_de_fin date NULL COMMENT 'Date de fin du cours, si elle est définie.',
17  Modalite_de_suivi varchar(10) NOT NULL COMMENT 'Modalité de suivi du cours, pouvant être en autonomie, en direct en présentiel ou en direct à distance.',
18  CONSTRAINT Cours_pk PRIMARY KEY (Id_Cours)
19 ) COMMENT 'Cours identifié par son intitulé.';
20
21 -- Table: Cours_Utilisateurs
22 CREATE TABLE Cours_Utilisateurs (
23   Cours_Id_Cours int NOT NULL COMMENT 'FK
24   Identifiant unique du cours.',
25   Utilisateurs_Id_Utilisateur int NOT NULL COMMENT 'FK
26   Identifiant unique de l''''utilisateur.',
27   CONSTRAINT Cours_Utilisateurs_pk PRIMARY KEY (Cours_Id_Cours,Utilisateurs_Id_Utilisateur)
28 );
29
30 -- Table: Examens
31 CREATE TABLE Examens (
32   Id_Cours int NOT NULL,
33   Titre varchar(30) NOT NULL COMMENT 'Titre de l''''examen.',
34   Contenu_textuel_longtext NOT NULL COMMENT 'Contenu textuel de l''''examen.',
35   CONSTRAINT Examens_pk PRIMARY KEY (Id_Cours)
36 ) COMMENT 'Examens portant sur certaines parties d''''un cours, identifié par son titre.';
37
38 -- Table: Inscriptions_Cours
39 CREATE TABLE Inscriptions_Cours (
40   Id_Inscription int NOT NULL,
41   Id_Cours int NOT NULL COMMENT 'FK
42   Identifiant unique du cours.',
43   Id_Utilisateur int NOT NULL COMMENT 'FK
44   Identifiant unique de l''''utilisateur.',
45   Appreciation_Note int NOT NULL COMMENT 'Note attribuée par l''''étudiant au cours, généralement sur une échelle de 1 à 5.',
46   Appreciation_Commentaire varchar(100) NULL COMMENT 'Commentaire optionnel fourni par l''''étudiant sur le cours.',
47   Paiement_Date_Transaction date NOT NULL COMMENT 'Date de paiement ',
48   Paiement_Montant decimal(6,2) NOT NULL COMMENT 'Montant de transaction ',
49 )
```

Une fois les erreurs corrigées, nous avons écrit le code d'insertion des données pour contrôler les données de notre base (voir database\_data.sql). Ainsi, nous avons pu générer des données cohérentes qui soient systématiquement les mêmes en cas de régénération de notre base de zéro. Nous avons utilisé faker au départ, mais n'avons pas gardé cette solution à cause de certains bugs non résolus lors de l'utilisation. Dans sa version finale, notre script répond à priori entièrement à la consigne : un jeu de données cohérent et couvrant toutes les tables a été ajouté.

```
1 -- phpMyAdmin SQL Dump
2 -- version 5.2.1
3 -- https://www.phpmyadmin.net/
4
5 -- Serveur: localhost
6 -- Temps de génération: 22-03-2024 à 14:15:23
7 -- Version du serveur: 10.4.28-MariaDB
8 -- Version de PHP: 8.2.4
9
10 SET SQL_MODE = "NO_AUTO_VALUE_ON_ZERO";
11 START TRANSACTION;
12 SET time_zone = "+00:00";
13
14
15 /*40101 SET @OLD_CHARACTER_SET_CLIENT=@@CHARACTER_SET_CLIENT */;
16 /*40101 SET @OLD_CHARACTER_SET_RESULTS=@@CHARACTER_SET_RESULTS */;
17 /*40101 SET @OLD_COLLATION_CONNECTION=@@COLLATION_CONNECTION */;
18 /*40101 SET NAMES utf8mb4 */;
19
20 --
21 -- Base de données: 'test2'
22 --
23
24 --
25 -- Volcado de datos para la tabla 'Utilisateurs'
26 --
27
28 INSERT INTO `Utilisateurs` (`Id_Utilisateur`, `Nom`, `Date_inscription`) VALUES
29 (1, 'Dubois', '2022-08-24'),
30 (2, 'Chen', '2020-04-04'),
31 (3, 'Girard', '2018-05-04'),
32 (4, 'Brown', '2018-11-03'),
33 (5, 'Lefebvre', '2015-12-14'),
34 (6, 'Bernard', '2012-08-17'),
35 (7, 'Moreau', '2012-02-05'),
36 (8, 'Félix', '2015-04-11'),
37 (9, 'Dorand', '2021-02-17'),
38 (10, 'Fournier', '2019-10-04'),
39 (11, 'Garcia', '2017-10-29'),
40 (12, 'Lambert', '2023-04-16'),
41 (13, 'Lacroix', '2017-10-29'),
42 (14, 'Martin', '2019-11-06'),
43 (15, 'Vandenberg', '2012-09-13'),
44 (16, 'Simon', '2011-03-05'),
45 (17, 'Fontaine', '2014-11-10'),
46 (18, 'Morel', '2022-02-28'),
47 )
```

Une fois les erreurs corrigées, nous avons rédigé le scripts de nos contraintes, triggers et procédures, ainsi qu'une batterie de tests (voir database\_constraints.sql). Dans sa version finale, notre script répond à priori entièrement à la consigne : toutes les contraintes, triggers et procédures implicites et explicites ont été implémentées et sont fonctionnelles (voir tests).

```

1 -- POUR CONSULTER LES REQUÊTES PERMETTANT DE TESTER LES DIFFÉRENTES CONTRAINTES, SE RÉFÉREZ LIGNE 536
2
3 ----- CONTRAINTES -----
4
5 -- Contraintes pour projet_ginoux_flores
6
7 -- Table Inscriptions_Cours
8
9 -- La note d'appréciation des cours doit être entre 1 et 5
10 ALTER TABLE Inscriptions_Cours
11 ADD CONSTRAINT enoandement_appreciation_note
12 CHECK (Appreciation_Note BETWEEN 1 AND 5);
13
14 -- Table Cours
15
16 -- L'intitulé d'un cours ne peut pas être vide
17 ALTER TABLE Cours
18 ADD CONSTRAINT check_intitule_not_empty
19 CHECK (Intitule IS NOT NULL AND Intitule != '');
20
21 -- L'intitulé d'un cours ne peut pas être vide
22 ALTER TABLE Cours
23 ADD CONSTRAINT check_intitule_length
24 CHECK (LENGTH(Intitule) <= 50);
25
26 -- L'attribut "Prix" de Cours doit être entre 0 et 1000
27 ALTER TABLE Cours
28 ADD CONSTRAINT check_prix
29 CHECK (Prix BETWEEN 0 AND 1000);
30
31 -- L'attribut "Modalité de suivi" de Cours ne peut valoir que "autonomie" ou "en_session"
32 ALTER TABLE Cours
33 ADD CONSTRAINT check_modalite_suivi_valides
34 CHECK (Modalite_de_suivi IN ('autonomie', 'en_session'));
35
36 -- L'attribut "Nombre de chapitres" est inférieur ou égal à "nombre de parties"
37 ALTER TABLE Cours
38 ADD CONSTRAINT check_nb_chapitres
39 CHECK (Nombre_de_chapitres <= Nombre_de_parties);
40
41 -- La description d'un cours ne peut pas être vide
42 ALTER TABLE Cours
43 ADD CONSTRAINT check_description_not_empty
44 CHECK (Description IS NOT NULL AND Description != '');
45
46 -- La description d'un cours ne peut pas être vide
47 ALTER TABLE Cours
48 ADD CONSTRAINT check_description_length
49 CHECK (LENGTH(Description) <= 1000);
50
51 -- Les pré-requis d'un cours ne peuvent pas être vides
52 ALTER TABLE Cours
53 ADD CONSTRAINT check_pre_requis_not_empty
54 CHECK (Pre_requis IS NOT NULL AND Pre_requis != '');
55
56 -- Les pré-requis d'un cours ne peuvent pas être vides
57 ALTER TABLE Cours
58 ADD CONSTRAINT check_pre_requis_length
59 CHECK (LENGTH(Pre_requis) <= 1000);
60

```

Une fois les erreurs corrigées, nous avons rédigé les requêtes de la partie 4 (voir database\_queries.sql). Dans sa version finale, notre script répond à priori entièrement à la consigne : toutes les requêtes ont été rédigées et fonctionnent.

```

1 -- 1.- Afficher les cours par ordre de popularité :
2
3 -- a) Par nombre d'utilisateurs inscrits
4 SELECT C.Id_Cours, C.Intitule AS Titre_Cours, COUNT(CU.Utilisateurs_Id_Utilisateur) AS Nombre_Inscriptions
5 FROM Cours C
6 LEFT JOIN Cours_Utilisateurs CU ON C.Id_Cours = CU.Cours_Id_Cours
7 GROUP BY C.Id_Cours, C.Intitule
8 ORDER BY Nombre_Inscriptions DESC;
9
10 -- b) Par meilleure évaluation (notes sur 5 données par les utilisateurs d'un cours)
11 SELECT
12 C.Id_Cours,
13 C.Intitule AS Titre_Cours,
14 AVG(IC.Appreciation_Note) AS Moyenne_Evaluation,
15 COUNT(IC.Appreciation_Note) AS Nombre_Evaluations
16 FROM
17 Cours C
18 LEFT JOIN
19 Inscriptions_Cours IC ON C.Id_Cours = IC.Id_Cours
20 WHERE
21 IC.Appreciation_Note IS NOT NULL
22 GROUP BY
23 C.Id_Cours,
24 C.Intitule
25 ORDER BY
26 Moyenne_Evaluation DESC;
27
28 -- 2.- Pour un cours donné, afficher la liste des utilisateurs :
29 -- a) Qui ont terminé le cours (toutes les parties ont été marquées comme validées)
30
31 -- 3.- Afficher la liste des utilisateurs par ordre de dépenses (les utilisateurs qui ont dépensé le plus d'argent en achetant des cours payants. On doit voir le montant dépensé dans le résultat de la requête)
32 SELECT
33 U.Id_Utilisateur,
34 U.Nom AS Nom_Utilisateur,
35 SUM(IC.Paiement_Montant) AS Montant_Depense
36 FROM
37 Utilisateurs U
38 INNER JOIN
39 Inscriptions_Cours IC ON U.Id_Utilisateur = IC.Id_Utilisateur
40 INNER JOIN
41 Cours C ON IC.Id_Cours = C.Id_Cours
42 WHERE
43 C.Prix > 0 -- seulement cours payants
44 GROUP BY
45 U.Id_Utilisateur,
46 U.Nom
47 ORDER BY
48 Montant_Depense DESC;
49

```

### **Difficultés rencontrées et travail de groupe :**

Les principales difficultés que nous avons rencontrées étaient durant la phase de conception. Le reste du projet dépendant directement de celle-ci, nous avons passé beaucoup de temps à re-modéliser notre base de données, ainsi qu'à mettre à jour les scripts d'insertion et de contraintes, avant d'avoir une structure satisfaisante.

Nous avons aussi rencontré certaines difficultés à mettre en place DBeaver, uniquement à l'initialisation du logiciel.

Nous avons utilisé les outils suivant :

- Vertabelo pour la modélisation des modèles Entité-Association et physiques, ainsi que pour le partage des scripts de création, d'insertion, de contraintes et de requêtes.
- Trello pour l'organisation et la répartition du travail, le suivi d'avancement de projet, ainsi que pour les comptes rendus toutes les deux semaines.
- Whatsapp pour les échanges informels sur l'avancement du projet.

